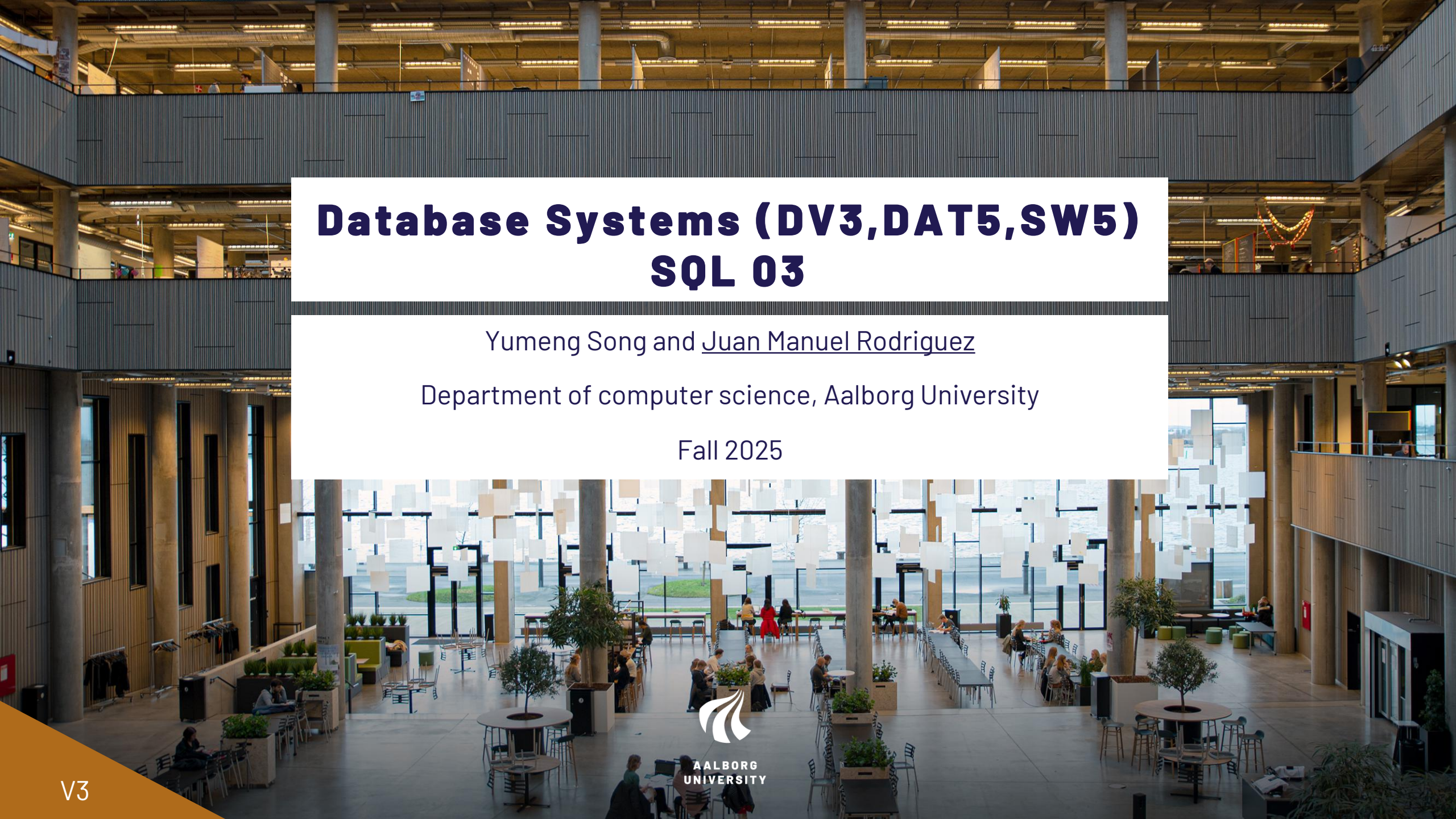


Database Systems (DV3,DAT5,SW5)
SQL 03

Yumeng Song and Juan Manuel Rodriguez

Department of computer science, Aalborg University

Fall 2025

AALBORG UNIVERSITY

V3

Outline lecture 01

Core concepts ★

- Database & DBMS
- Data model & Schema
- Relational model
- Logical / Physical separation
- Relation / Table / Attributes / Columns / Tuples /Rows
- Cardinality / Arity
- Primary keys / Foreign keys
- Procedural / Declarative Language

Intro to SQL

- Sets and Bags ★
- DDL / DML / DQL / TCL / DCL

Data Definition Language [part 1] ★

- CREATE (as query / with data)
- Types: char/ varchar/text + default
- Integrity constraints: primary key, unique, not null / foreign key , check
- Delete table / truncate
- Alter Table

Data Manipulation Language [part 1] ★

- Insert
- Delete (+ where)

Data Query Language [part 1] ★

- SELECT / FROM / WHERE
- Conditions
- PROJECTION & DISTINCT
- CROSS PRODUCT & JOIN
- THETA / NATURAL JOIN
- Renaming
- Set operations

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:
Chapters 3 and 4, Section 5.4
- from Database System Concepts 7th edition ; Silberschatz et al.

Outline lecture 02

Data Manipulation Language [part 2] ★

- Insert (+ select)
- Update (+where)
- Referential Integrity

Data Definition Language [part 2]

- CREATE (as query / with data) ★
- Sequence numbers
- Cascade / set NULL / set default ★

Data Query Language [part 2] ★

- LIMIT
- Nested queries / Uncorrelated subqueries
- Conditions IN / NOT IN / EXISTS / ALL / ANY
- INNER / OUTER / LEFT JOIN
- NULLs
- WITH statement

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:
Chapters 3 and 4, Section 5.4
- from Database System Concepts 7th edition ; Silberschatz et al.

Outline of this lecture

Data Query Language [part 3]

- Aggregates ★
- Aggregates & subqueries ★
- Grouping ★
- NULLs with Joins, aggregates, grouping ★
- Recursion

Data Definition Language [part 3]

- views + updatable views + materialized views ★

Data Control Language

- Grant/revoke

Transaction Control Language

- begin/commit/rollback ★

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

SQL – Not only SELECT queries

Data Definition Language	Data Manipulation Language	Data Query Language	Transaction Control Language	Data Control Language

`CREATE TABLE`	`INSERT INTO`
	`DELETE FROM`
	`UPDATE`
		`SELECT FROM`
		`WHERE`
			`BEGIN;`
			`COMMIT;`
			`ROLLBACK;`
				`GRANT ON TO`

DDL	DML	DQL	TCL	DCL

Today we complete the picture

Find the best sports team

- Given the table **team (id, name, points) , find the best team and how many points they have**
- **The best team has the highest number of points**

```
SELECT name, points
FROM team
ORDER BY points DESC
LIMIT 1;
```

WRONG

Multiple teams can have the same amount of points

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Find the best sports team

- Given the table **team (id, name, points) , find the best team and how many points they have**
- **The best team has the highest number of points**

```
SELECT name, points
FROM team
WHERE points >= ALL (SELECT points
                     FROM team);
```

Multiple teams can have the same amount of points

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Aggregate function

```
SELECT MAX(points)
FROM team
```

This is not enough to solve the query in the previous slide

- Aggregate functions compute new values for a column, e.g., the sum or the average of all values in a column
- Aggregate functions: AVG, MAX, MIN, COUNT, SUM

```
SELECT SUM(ects)
FROM course
WHERE taughtby = 2125;
```

```
SELECT AVG(grade)
FROM grades
WHERE courseid = 5001;
```


Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Aggregate example

- Aggregate functions: AVG, MAX, MIN, COUNT, SUM

worker

empid	name	months	salary
5001	Bob	4	20000
5041	Anne	4	21000
5043	Alex	3	15000
5049	Jane	2	10000
4052	Lars	4	15000
5052	Mette	3	40000
5216	Rikke	2	10000

```
SELECT SUM(salary) AS total
FROM worker
WHERE months > 3;
```

total
56000

```
SELECT AVG(salary) AS rate
FROM worker
WHERE months = 4;
```

rate
18666.666

```
SELECT SUM(salary)/COUNT(salary) AS rate
FROM worker
WHERE months = 4;
```

```
SELECT MAX(salary) AS x
FROM worker
WHERE months > 3;
```

x
21000

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Aggregate example (II)

- Number of different workers

```
SELECT COUNT(*) AS total
FROM worker
```

total
7

```
SELECT COUNT(empid) AS total
FROM worker
```

total
7

```
SELECT COUNT(42) AS total
FROM worker
```

total
7

```
SELECT SUM(1) AS total
FROM worker
```

total
7

worker

empid	name	months	salary
5001	Bob	4	20000
5041	Anne	4	21000
5043	Alex	3	35000
5049	Jane	2	10000
4052	Lars	4	15000
5052	Mette	3	40000
5216	Rikke	2	10000

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Aggregate example -NULLs

- Number of different salary or months

```
SELECT COUNT(DISTINCT months) AS m
FROM worker;
```

m
3

```
SELECT COUNT(months) AS m
FROM worker;
```

m
6

```
SELECT MAX(months) AS m
FROM worker;
```

m
4

```
SELECT MIN(months) AS m
FROM worker;
```

m
2

worker

empid	name	months	salary
5001	Bob	4	20000
5041	Anne	4	21000
5043	Alex	3	35000
5049	Jane	2	10000
4052	Lars	**NULL**	15000
5052	Mette	3	40000
5216	Rikke	2	**NULL**

```
SELECT DISTINCT months
FROM worker
```

months
4
3
2
NULL

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Handling NULLs with care!

student

studid	name	semester
24002	Xenokrates	⊥
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	⊥
27550	Schopenhauer	6
28106	Carnap	⊥
29120	Theophrastos	2
29555	Feuerbach	2

```
SELECT COUNT(semester) AS c
FROM student;
```

c
5

```
SELECT COUNT(*) AS c
FROM student;
```

c
8

```
SELECT COUNT(DISTINCT semester) AS c
FROM student;
```

c
4

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Handling NULLs with care!

student

studid	name	semester
24002	Xenokrates	⊥
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	⊥
27550	Schopenhauer	6
28106	Carnap	⊥
29120	Theophrastos	2
29555	Feuerbach	2

```
SELECT COUNT(semester) AS c
FROM student;
```

c
5

not the same!

```
SELECT COUNT(*) AS c
FROM student;
```

c
8

```
SELECT COUNT(DISTINCT semester) AS c
FROM student;
```

c
4

```
SELECT  COUNT(*) AS c
FROM  (SELECT DISTINCT semester FROM student) AS st;
```

c
5

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

FIND the MIN!

Find the students in the lowest semester

```
SELECT studid, MIN(semester)
FROM student;
```

Incorrect statement

```
SELECT studid, semester
FROM student
ORDER BY semester ASC
FETCH FIRST 1 ROWS ONLY;
```

Incorrect statement

```
SELECT studid, semester
FROM student
WHERE semester = (SELECT MIN(semester)
                  FROM student);
```

Correct statement

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

FIND the MIN! (II)

Find the lowest semester in the DB

```
SELECT MIN(semester)
FROM student;
```

```
SELECT semester
FROM student
ORDER BY semester ASC
FETCH FIRST 1 ROWS ONLY;
```

Inefficient! don't do this!

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Complex example

The subquery produces a single value and can be used to compare with other single value expressions in the WHERE clause

List all the students that have grades above the average

```
SELECT DISTINCT studid, name, semester
FROM student NATURAL JOIN grades
WHERE grade > (SELECT AVG(grade) FROM grades);
```

AVG(grade)
1.6666666666666667

Uncorrelated subquery

student

studid	name	semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

grades

studid	courseid	empid	grade
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2

studid	name	semester
25403	Jonas	12
27550	Schopenhauer	6

Aggregated and Subqueries - SELECT

You can use Aggregations in subqueries for advanced filtering

- Professor's id, name, and total number of ects taught

```
SELECT id, name, (SELECT SUM(ects)
                  FROM course
                  WHERE taughtby = id) AS teachingLoad
FROM professor;
```

This subquery is evaluated once for each result tuple

Correlated subquery!

GROUPING RESULTS (I) - BASICS

Instead of computing 1 aggregation for the entire table, I want multiple aggregation results

For **every** professor, the SUM of ECTS

```
SELECT SUM(ects) AS sumECTS
FROM course
GROUP BY taughtby;
```

- Aggregate function is computed per group
- All tuples with the same value for column **taughtby** form a group
- The sum is computed for each group

course

courseid	title	ects	taughtby
5001	Basics	4	2137
4630	Constructive Criticism	4	2137
5041	Ethics	4	2125
5049	DBS	1	2125
4052	Logics	4	2125
5043	Theory of Cognition	3	2126
5052	Theory of Science	3	2126
5216	Bioethics	3	2126
5259	Advanced Algorithms	2	2133
5022	Belief and Knowledge	2	2134

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

GROUPING RESULTS (II)

```
SELECT SUM(ects) AS sumECTS
FROM course
GROUP BY taughtby;
```

```
SELECT SUM(ects) AS sumECTS, taughtby
FROM course
GROUP BY taughtby;
```

course

courseid	title	ects	taughtby	
5001	Basics	4	2137	G1
4630	Constructive Criticism	4	2137	
5041	Ethics	4	2125	G2
5049	DBS	1	2125	
4052	Logics	4	2125	
5043	Theory of Cognition	3	2126	G3
5052	Theory of Science	3	2126	
5216	Bioethics	3	2126	
5259	Advanced Algorithms	2	2133	G4
5022	Belief and Knowledge	2	2134	G5

	sumECTS	taughtby
G1	8	2137
G2	9	2125
G3	9	2126
G4	2	2133
G5	2	2134

Attributes in the SELECT must be compatible with those in the GROUP BY!
(see next slides)

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

GROUPING RESULTS (III)

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4

```
SELECT rank, COUNT(id), name
FROM professor
GROUP BY rank;
```

Incorrect statement

- SQL generates **one result tuple per group**
- All column references in the SELECT clause **must** either be listed in the GROUP BY clause or involved only in aggregate functions

rank	COUNT(id)	name
C2	1	
C3	2	
C4	3	

```
SELECT rank, COUNT(id)
FROM professor
GROUP BY rank;
```

Correct statement

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

GROUPING RESULTS (IV)

professor

id	name	rank
2125	Socrates	C4
2126	Russel	C4
2128	Russel	C2
2133	Popper	C3
2134	Augustinus	C3
2137	Kant	C4
2140	Kant	C4

Q1
```
SELECT COUNT(*)
FROM professor
GROUP BY rank;
```
Correct statement!

Q2
```
SELECT name, rank
FROM professor
GROUP BY name, rank;
```
Correct statement!
same of just using DISTINCT without GROUP BY

Q3
```
SELECT rank, COUNT(*)
FROM professor
GROUP BY name, rank;
```
Correct statement!

Q3

rank	COUNT(*)
C2	1
C3	1
C3	1
C4	1
C4	1
C4	1
C4	2

Q4
```
SELECT DISTINCT name, rank
FROM professor
GROUP BY name, rank;
```
Correct but Meaningless statement!
DISTINCT and GROUP BY are doing the same work twice

Q5
```
SELECT DISTINCT rank
FROM professor
GROUP BY name, rank;
```
Correct statement!
but not very useful, what is this doing?

Q6
```
SELECT COUNT(DISTINCT rank)
FROM professor
GROUP BY name;
```
Correct statement!

NEW

Aggregated and Subqueries (II) – FROM

Nesting Aggregate functions does not work even with group by!

- Compute the average teaching load across professors:
  the average of the sum of ECTS they teach

```
SELECT AVG(SUM(ects)) AS result
FROM course c
GROUP BY taughtby;
```

Incorrect statement

```
SELECT AVG(teachingLoad) AS result
FROM (SELECT SUM(ects) AS teachingLoad
      FROM course c
      GROUP BY taughtby) as r;
```

Correct statement

FILTERING GROUPS

Only return a subset of groups satisfying some condition

Return professor with a SUM of ECTS > 8

```
SELECT SUM(ects) AS sumECTS
FROM course
GROUP BY taughtby
HAVING SUM(ects) > 8;
```

course

courseid	title	ects	taughtby
5001	Basics	4	2137
4630	Constructive Criticism	4	2137
5041	Ethics	4	2125
5049	DBS	1	2125
4052	Logics	4	2125
5043	Theory of Cognition	3	2126
5052	Theory of Science	3	2126
5216	Bioethics	3	2126
5259	Advanced Algorithms	2	2133
5022	Belief and Knowledge	2	2134

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

FILTERING GROUPS – Example Computation

Only return a subset of groups satisfying some condition

```
SELECT taughtby, name, SUM(ects)
FROM course, professor
WHERE taughtby = id AND rank = 'C4'
GROUP BY taughtby, name HAVING AVG(ects) > 3;
```

intermediate join result

courseid	title	ects	taughtby	id	name	rank	AVG(ects)	AVG(ects) > 3
5041	Ethics	4	2125	2125	Socrates	C4		
5049	DBS	2	2125	2125	Socrates	C4	3.33333	true
4052	Logics	4	2125	2125	Socrates	C4		
5043	Theory of Cognition	3	2126	2126	Russel	C4		
5052	Theory of Science	3	2126	2126	Russel	C4	3	false
5001	Basics	4	2137	2137	Kant	C4		
4630	Constructive Criticism	4	2137	2137	Kant	C4	4	true

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

FILTERING GROUPS – Example Computation

Only return a subset of groups satisfying some condition

```
SELECT taughtby, name, SUM(ects)
FROM course, professor
WHERE taughtby = id AND rank = 'C4'
GROUP BY taughtby, name HAVING AVG(ects) > 3;
```

Variables in the HAVING clause must appear in the GROUP BY clause or be used in an aggregate function

intermediate group-by result

courseid	title	ects	taughtby	id	name	rank	SUM(ects)
5041	Ethics	4	2125	2125	Socrates	C4	10
5049	DBS	2		2125		C4	
4052	Logics	4		2125		C4	
5001	Basics	4	2137	2137	Kant	C4	8
4630	Constructive Criticism	4		2137		C4	

final result

taughtby	name	SUM(ects)
2125	Socrates	10
2137	Kant	8

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

FILTERING GROUPS – Advanced (I)

id	name	rank	salary
2125	Socrates	C4	100
2126	Russel	C4	100
2134	Augustinus	C3	200
2137	Kant	C4	200
2140	Magnus	C2	200
2141	Eco	C9	1000

id	title	ects	students	by
1	Basic	4	20	2137
2	Ethics	4	10	2125
3	Theory Cogn	3	50	2126
4	CS 101	2	40	2134
5	Math	8	20	2137
6	Morals	3	20	2140
7	Algebra	3	20	2126
8	Algos	6	20	2125
9	OS	2	100	2125

```
SELECT p.id, p.name, SUM(c.students)
FROM professor p
     JOIN course c ON p.id = c.by
WHERE AVG(ects) > 2
GROUP BY p.id, p.name;
```

Incorrect statement

```
SELECT p.id, p.name, SUM(c.students)
FROM professor p
     JOIN course c ON p.id = c.by
GROUP BY p.id, p.name
HAVING AVG(ects) > 2;
```

Correct statement!

FILTERING GROUPS – Advanced (II)

id	name	rank	salary
2125	Socrates	C4	100
2126	Russel	C4	100
2134	Augustinus	C3	200
2137	Kant	C4	200
2140	Magnus	C2	200
2141	Eco	C9	1000

id	title	ects	students	by
1	Basic	4	20	2137
2	Ethics	4	10	2125
3	Theory Cogn	3	50	2126
4	CS 101	2	40	2134
5	Math	8	20	2137
6	Morals	3	20	2140
7	Algebra	3	20	2126
8	Algos	6	20	2125
9	OS	2	100	2125

```
SELECT p.id, p.name, SUM(c.students)
FROM professor p
     JOIN course c   ON p.id = c.by
GROUP BY p.id, p.name
HAVING c.ects > 3;
```

Incorrect statement

```
SELECT p.id, p.name, SUM(c.students)
FROM professor p
     JOIN course c   ON p.id = c.by
WHERE c.ects > 3
GROUP BY p.id, p.name;
```

Correct statement!

FILTERING GROUPS – Advanced (III)

id	name	rank	salary
2125	Socrates	C4	100
2126	Russel	C4	100
2134	Augustinus	C3	200
2137	Kant	C4	200
2140	Magnus	C2	200
2141	Eco	C9	1000

id	title	ects	students	by
1	Basic	4	20	2137
2	Ethics	4	10	2125
3	Theory Cogn	3	50	2126
4	CS 101	2	40	2134
5	Math	8	20	2137
6	Morals	3	20	2140
7	Algebra	3	20	2126
8	Algos	6	20	2125
9	OS	2	100	2125

```
SELECT p.id, p.name, SUM(c.students)
FROM professor p
     LEFT JOIN course c ON p.id = c.by
GROUP BY p.id, p.name
HAVING COUNT(*) < 2
```

Correct statement!

Note “Eco” does not teach courses, so what is the output of the two queries?

```
SELECT p.id, p.name, p.salary, SUM(c.students)
FROM professor p
     LEFT JOIN course c ON p.id = c.by
                AND c.ects > 2
GROUP BY p.id, p.name, p.salary
HAVING p.salary > 100
```

Correct statement!

For the curious reader
https://www.vldb.org/pvldb/vol15/p2613-guagliardo.pdf

Handling NULLs with care!

```
SELECT semester, COUNT(studid) as number
FROM student
GROUP BY semester;
```

student

studid	name	semester
24002	Xenokrates	⊥
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	⊥
27550	Schopenhauer	6
28106	Carnap	⊥
29120	Theophrastos	2
29555	Feuerbach	2

Number of students per semester

semester	number
⊥	3
12	1
10	1
6	1
2	2

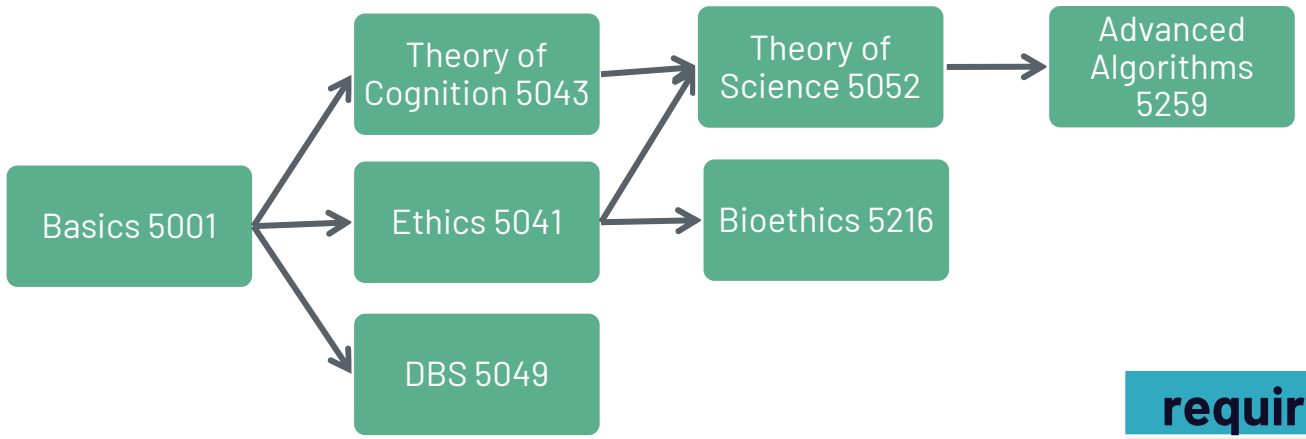
NULL is interpreted as an independent value and results in its own group

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

A simple case .... ?

Find the prerequisites to learn Theory of Science

```
SELECT r2.requirement
FROM requires r1, requires r2, course c
WHERE c.title = 'Theory of Science' AND
      c.courseid = r1.cid AND
      r1.requirement = r2.cid;
```



requirement
5001
5001

course

id	name
5259	Adv. Algo
5052	Theory of Science
5043	Theory of Cogn.
5216	Bioethics
5041	Ethics
5049	DBS
5001	Basics

requires

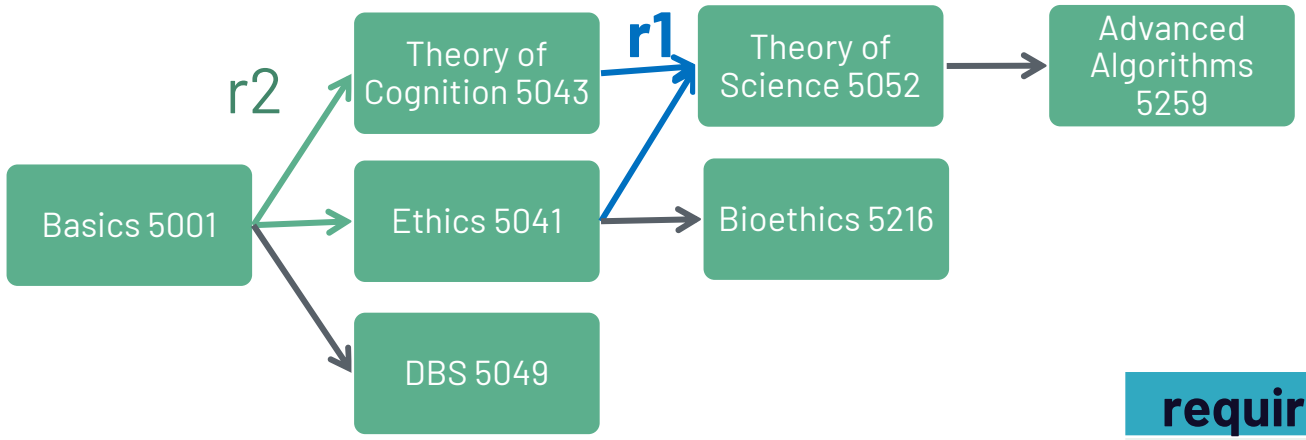
cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5049	5001
5041	5001

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

A simple case .... ?

Find the prerequisites to learn Theory of Science

```
SELECT r2.requirement
FROM requires r1, requires r2, course c
WHERE c.title = 'Theory of Science' AND
      c.courseid = r1.cid AND
      r1.requirement = r2.cid;
```



requirement
5001
5001

course

id	name
5259	Adv. Algo
5052	Theory of Science
5043	Theory of Cogn.
5216	Bioethics
5041	Ethics
5049	DBS
5001	Basics

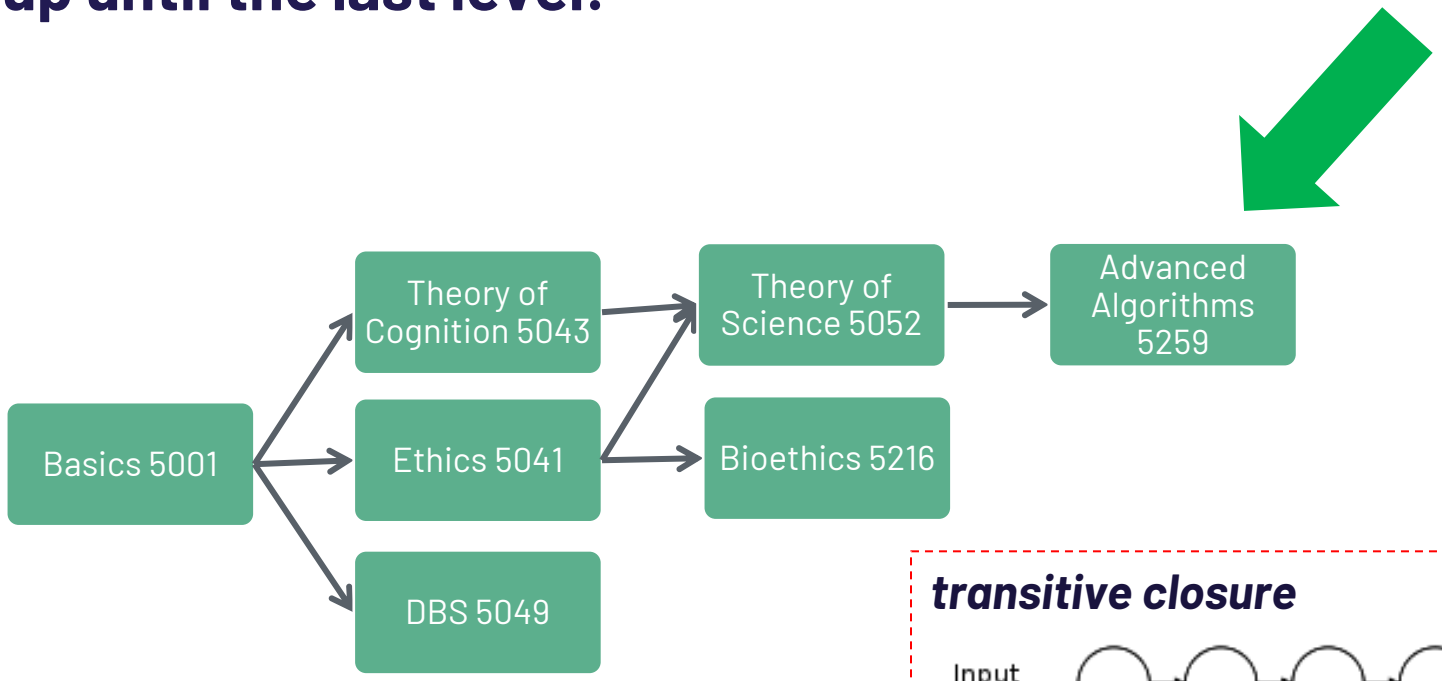
requires

cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5049	5001
5041	5001

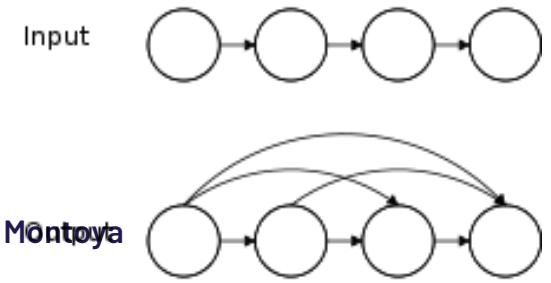
Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

A HARD case !

Given a course, find a list of ALL the requirements up until the last level!



transitive closure



course

id	name
5259	Adv. Algo
5052	Theory of Science
5043	Theory of Cogn.
5216	Bioethics
5041	Ethics
5049	DBS
5001	Basics

requires

cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5049	5001
5041	5001

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

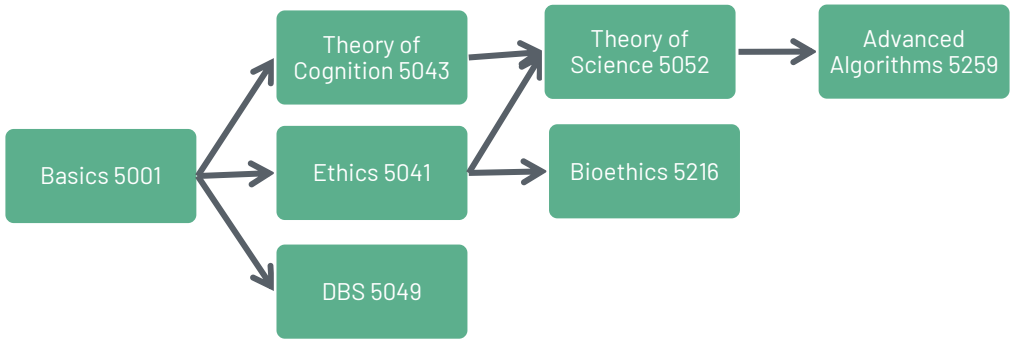
A HARD case : Intuition

Given a course, find a list of ALL the requirements up until the last level!

(step 1) Given the course, find the requirements, put them in a set

(step 2) for each one of these requirements, find its requirements, add them also to the set above

(step 3) for anything in that set, ... repeat step 2!



{ 5052 }
{ 5052 } U { 5043, 5041}
{ 5052, 5043, 5041} U { 5001, 5001}

course

id	name
5259	Adv. Algo
5052	Theory of Science
5043	Theory of Cogn.
5216	Bioethics
5041	Ethics
5049	DBS
5001	Basics

requires

cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

RECURSION: Solving the issue (I)

requires

cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

```
WITH RECURSIVE closure(curr, req) AS (
    SELECT cid, requirement
    FROM requires
UNION
    SELECT DISTINCT r.cid, c.req
    FROM requires r, closure c
    WHERE r.requirement = c.curr
)
SELECT *
FROM closure
WHERE curr = 5259
ORDER BY(curr, req) DESC;
```

Non-recursive part

Recursive part

Main query

curr	req
5259	5052
5259	5043
5259	5041
5259	5001

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

RECURSION: Solving the issue (*)

```
WITH RECURSIVE closure(curr, req) AS (
    SELECT cid, requirement
    FROM requires
UNION
    SELECT DISTINCT r.cid, c.req
    FROM requires r, closure c
    WHERE  r.requirement = c.curr
)
SELECT *
FROM closure
WHERE curr = 5259
ORDER BY(curr, req) DESC;
```

Non-recursive part

Recursive part

Main query

requires

cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

closure: non recursive -step 0

curr	req
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

closure: intermediate result step 0

curr	req
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

NEW

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

RECURSION: Solving the issue (*)

```
WITH RECURSIVE closure(curr, req) AS (
    SELECT cid, requirement
    FROM requires
UNION
    SELECT DISTINCT r.cid, c.req
    FROM requires r, closure c
    WHERE  r.requirement = c.curr
)
SELECT *
FROM closure
WHERE curr = 5259
ORDER BY(curr, req) DESC;
```

Non-recursive part

Recursive part

Main query

requires

cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

closure: recursive part step 0

curr	req
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

closure: intermediate result step 1

curr	req
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001
5259	5043
5259	5041
5052	5001
5216	5001
5052	5001

of these 2,
1 will be removed by UNION!

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

RECURSION: Solving the issue (**)

```
WITH RECURSIVE closure(curr, req) AS (
    SELECT cid, requirement
    FROM requires
UNION
    SELECT DISTINCT r.cid, c.req
    FROM requires r, closure c
    WHERE  r.requirement = c.curr
)
SELECT *
FROM closure
WHERE curr = 5259
ORDER BY(curr, req) DESC;
```

Non-recursive part

Recursive part

Main query

requires

cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

closure: recursive part step 1

curr	req
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001
5259	5043
5259	5041
5052	5001
5216	5001

closure: intermediate result step 1 + 2

curr	req
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001
5259	5043
5259	5041
5052	5001
5216	5001

5259	5001

the next iteration will not add new tuples

requires

cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

RECURSION: Solving the issue (II)

requires

cid	requirement
5259	5052
5052	5043
5043	5001
5216	5041
5052	5041
5041	5001
5049	5001

```
WITH RECURSIVE closure(curr, req, step) AS (
    SELECT cid, requirement, 0
    FROM requires
UNION
    SELECT DISTINCT r.cid, c.req, c.step+1
    FROM closure c, requires r
    WHERE c.curr = r.requirement
)
SELECT *
FROM closure
WHERE curr = 5259
ORDER BY(curr, req, step) DESC;
```

Non-recursive part

Recursive part

Main query

curr	req	step
5259	5052	0
5259	5043	1
5259	5041	1
5259	5001	2

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

RECURSION: The Structure of the query

Sum all numbers from 1 to 100

```
WITH RECURSIVE mytable(number) AS (
    VALUES(1)
UNION
    SELECT number + 1
    FROM mytable
    WHERE number < 100
)
SELECT sum(number)
FROM mytable;
```

Non-recursive part

Recursive part

It references mytable!

Main query

Most DBMS have a parameter that limits maximum recursion depth
... otherwise you can use conditions in the recursive part to impose a max-recursion-depth

Outline of this lecture

Data Query Language [part 3]

- Aggregates ★
- Aggregates & subqueries ★
- Grouping ★
- NULLs with Joins, aggregates, grouping ★
- Recursion

Data Definition Language [part 3]

- views + updatable views + materialized views ★

Data Control Language

- Grant/revoke

Transaction Control Language

- begin/commit/rollback

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

Outline of this lecture

Data Query Language [part 3]

- Aggregates ★
- Aggregates & subqueries ★
- Grouping ★
- NULLs with Joins, aggregates, grouping ★
- Recursion

Data Definition Language [part 3]

- views + updatable views + materialized views ★

Data Control Language

- Grant/revoke

Transaction Control Language

- begin/commit/rollback

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

Store the results of a query as a Table

Alternative to CTE (Temporary Tables)?

Create a table with the result of a query...

Find the total ECTS for each professor, save it to a table

```
CREATE teachingLoad AS(
  SELECT taughtby AS pid, SUM(ects) AS load
  FROM course
  GROUP BY taughtby
);
```

Query the table

```
SELECT MAX(load) AS m
FROM teachingLoad;
```

What happens when the data updates?

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126
5049	DBS	2	2125
4052	Logics	4	2125
5052	Theory of Science	3	2126
5216	Bioethics	2	2128
5259	Advanced Algorithms	2	2133
5022	Belief and Knowledge	2	2134
4630	Constructive Criticism	4	2137

teachingLoad

pid	load
2125	10
2126	6
2128	2
2133	2
2134	2
2137	8

Result

m
10

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

VIEWS: External Schema

Store the query as a View

course

courseid	title	ects	taughtby
5001	Basics	4	2137
5041	Ethics	4	2125
5043	Theory of Cognition	3	2126
5049	DBS	2	2125
4052	Logics	4	2125
5052	Theory of Science	3	2126
5216	Bioethics	2	2128
5259	Advanced Algorithms	2	2133
5022	Belief and Knowledge	2	2134
4630	Constructive Criticism	4	2137

Create a table with the result of a query...

Find the total ECTS for each professor,

save ***the query*** *as a* ***VIEW***

```
CREATE VIEW teachingLoad AS(
  SELECT taughtby AS pid, SUM(ects) AS load
  FROM course
  GROUP BY taughtby
);
```

Query the view

```
SELECT MAX(load) AS m
FROM teachingLoad;
```

What happens when the data updates?

teachingLoad

pid	load
2125	10
2126	6
2128	2
2133	2
2134	2
2137	8

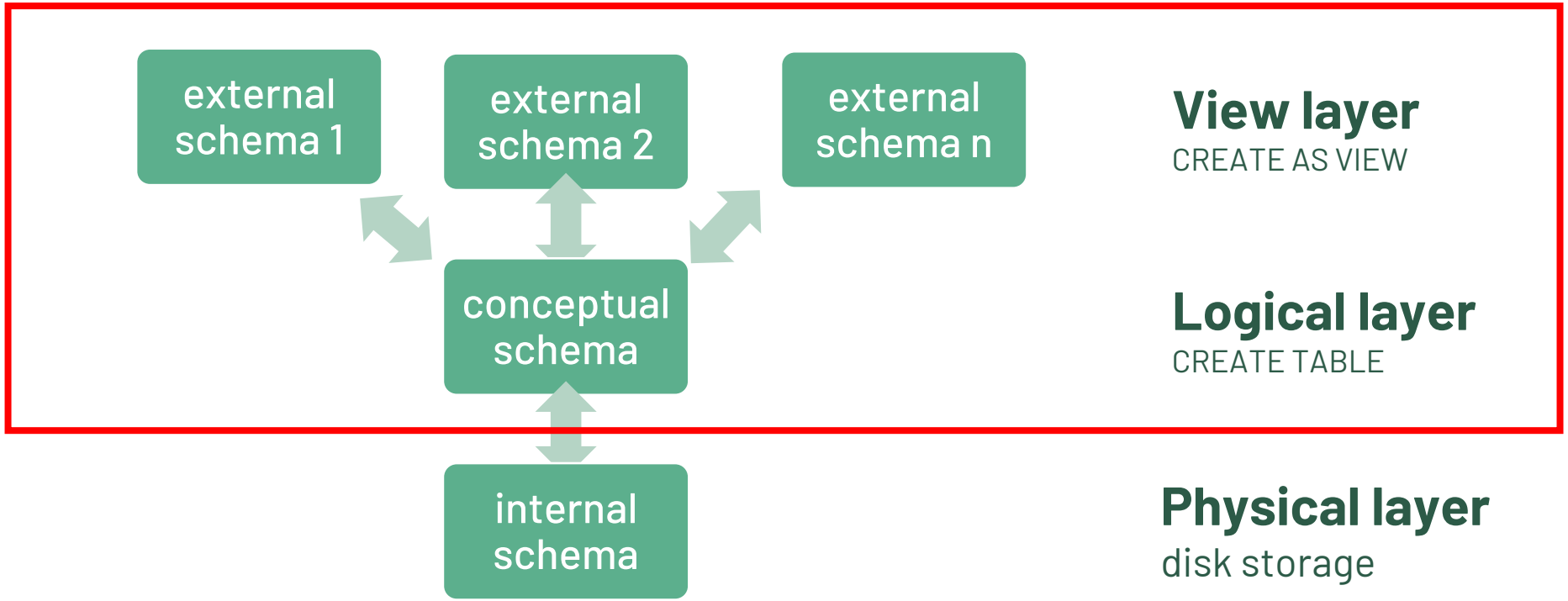
Result

m
10

AALBORG UNIVERSITY

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

DBMS Abstraction Layers



A DBMS ensures:

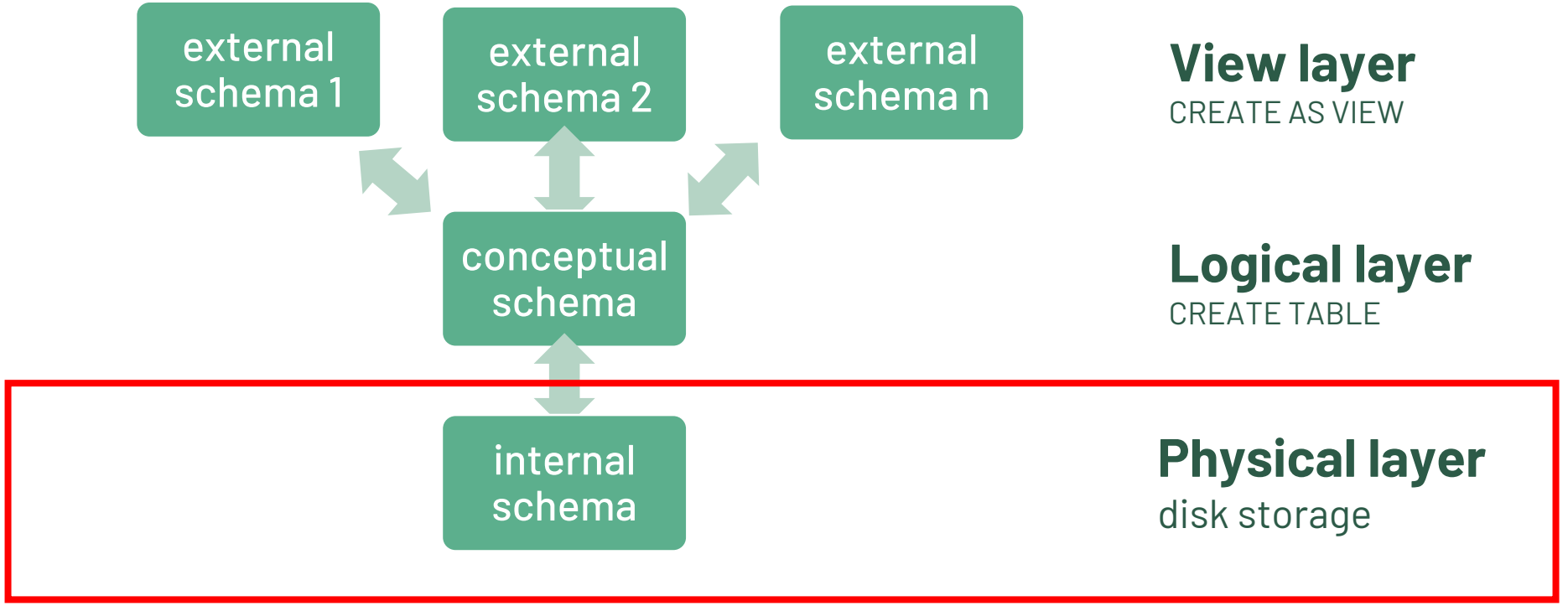
- Physical data independence: changes on the physical layer have no influence on the logical layer

Ideally we would like to ensure:

- Logical data independence: changes on the logical layer have no influence on the view layer

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

DBMS Abstraction Layers



A DBMS ensures:

- Physical data independence: changes on the physical layer have no influence on the logical layer

Ideally we would like to ensure:

- Logical data independence: changes on the logical layer have no influence on the view layer

VIEWS: Example (I)

course(courseid, title, ects, taughtby→professor)
professor(id, name, rank, office)

```
CREATE VIEW profsAndTheirCourses AS
    SELECT c.title, p.name
    FROM professor p, course c
    WHERE p.id = c.taughtby;

SELECT * FROM profsAndTheirCourses;
```

title	name
Basics	Kant
Ethics	Socrates
Theory of Cognition	Russel
DBS	Socrates
Logics	Socrates
Theory of Science	Russel
Bioethics	Russel
Advanced Algorithms	Popper
Belief and Knowledge	Augustinus
Constructive Criticism	Kant

- **A view is the result set of a stored query** on the data
- A view does not form part of the physical schema: as a result, **it is a virtual table computed dynamically** from data.
- **Changes** applied to the data in a relevant underlying table **are automatically reflected in the data** shown in subsequent invocations of the view.

VIEWS: Example (II)

student(studid, name, semester)

takes(studid→student, courseid→course)

course(courseid, title, ects, taughtby→professor)

```
CREATE VIEW ectsPerStud AS
    SELECT name, studid, SUM(ects) AS sum
    FROM student NATURAL JOIN takes NATURAL JOIN course
    GROUP BY name, studid;

SELECT sum FROM ectsPerStud;
```

Views can be used to represent derived attributes (ER diagram)

VIEWS:

- Views can represent **a subset of the data** contained in a table.
- Views can **join and simplify** multiple tables into a single virtual table.
- Views can act as **aggregated tables**, where the database engine aggregates data (sum, average, etc.) and presents the calculated results as part of the data.
- Views can **hide the complexity** of data.
- Views take very little space to store; **the database contains only the definition of a view, not a copy of all the data that it presents.**
- Depending on the SQL engine used, views can provide extra security.

Altering VIEWS (I)

- REPLACE VIEW expects the same columns (same order, same types)

```
CREATE OR REPLACE VIEW profsAndTheirCourses AS
    SELECT c.title, p.name
    FROM professor p, course c
    WHERE p.id = c.taughtby;
```


Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Altering VIEWS (II)

- Alternative to REPLACE VIEW: delete the view and recreate it afterwards

```
DROP VIEW profsAndTheirCourses ;

CREATE VIEW profsAndTheirCourses AS
    SELECT c.title, p.name
    FROM professor p, course c
    WHERE p.id = c.taughtby;
```


Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Updating VIEWS (I)

professor

id	name	rank	office
2125	Socrates	C4	226

```
CREATE VIEW howTough AS
    SELECT id, AVG(grade) AS avgGrade
    FROM grades
    GROUP BY id;
```

howTough

id	avgGrade
2125	3
2126	2
2137	2

grades

studid	courseid	id	grade
24002	5001	2126	3
28106	5001	2126	1
25403	5041	2125	2
27550	5041	2125	4
27550	4630	2137	2

This view is not updatable!!

```
UPDATE howTough
    SET avgGrade = 1.0
    WHERE id = (SELECT id
                FROM professor
                WHERE name = 'Socrates');
```

```
INSERT INTO howTough
VALUES (2921, 5.0);
```

How to adapt the grades in the original table? We cannot...

What tuples shall be inserted into the original table?

Updatable views?

course(courseid, title, ects, taughtby→professor)
professor(id, name, rank, office)

```
CREATE VIEW profsAndTheirCourses AS
    SELECT c.title, p.name
    FROM professor p, course c
    WHERE p.id = c.taughtby;
```

```
CREATE VIEW taughtCourses AS
    SELECT courseid, title
    FROM course
    WHERE taughtby IS NOT NULL;
```

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

A view is updatable if...

- The FROM clause has only one table
- The SELECT clause contains **only attribute names of the table** (no aggregates, no distinct, no expressions)
- Attributes **not** included in the SELECT clause can be set to **null**
- Does not include GROUP BY or HAVING clause

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Updatable views? (example 1)

course(courseid, title, ects, taughtby→professor)
professor(id, name, rank, office)

```
CREATE VIEW profsAndTheirCourses AS
    SELECT c.title, p.name
    FROM professor p, course c
    WHERE p.id = c.taughtby;
```

✘ Multiple tables in the FROM

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Updatable views? (example 2)

course(courseid, title, ects, taughtby→professor)
professor(id, name, rank, office)

```
CREATE VIEW profsAndTheirCourses AS
    SELECT c.taughtby, COUNT(c.courseid)
    FROM  course c
    WHERE c.ects > 2
    GROUP BY c.taughtby;
```

✘ Aggregates in SELECT

contains GROUP BY

```
CREATE VIEW profsAndTheirCourses AS
    SELECT DISTINCT c.taughtby, c.ects
    FROM  course c
    WHERE c.ects > 2;
```

✘ DISTINCT in SELECT

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Updatable views? (example 3)

course(courseid, title, ects, taughtby→professor)
professor(id, name, rank, office)

```
CREATE TABLE professor(
    id integer PRIMARY KEY,
    name varchar(10) NOT NULL,
    rank char(2) NOT NULL,
    office varchar(10)
);
```

```
CREATE VIEW highRankProfs AS
    SELECT p.id, p.name
    FROM  professor p
    WHERE p.rank > 'C2';
```

✘ p.rank is missing from the SELECT but is set to NO NULL

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Updatable views? (example 4)

course(courseid, title, ects, taughtby→professor)
professor(id, name, rank, office)

```
CREATE TABLE professor(
    id integer PRIMARY KEY,
    name varchar(10) NOT NULL,
    rank char(2) NOT NULL,
    office varchar(10)
);
```

```
CREATE VIEW highRankProfs AS
    SELECT p.id, p.name, p.rank
    FROM  professor p
    WHERE p.rank > 2;
```

✓ **NO multiple tables in FROM**

NO DISTINCT or AGGREGATES or GROUP BY

p.office is missing from the SELECT
but it is allowed to be NULL

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Materialized views

- (Dynamic) views represent a **macro** of a query and their result is computed **when used**
- The query result of a materialized view is **pre-computed and stored!**
- The computational load of materialized views is **before** any queries are executed

Which one is the better choice?

It depends: there is a trade-off

runtime vs. update frequency

```
CREATE MATERIALIZED VIEW
profsAndTheirCourses AS
    SELECT c.title, p.name
    FROM professor p, course c
    WHERE p.id = c.taughtby;
```


Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Outline of this lecture

Data Query Language [part 3]

- Aggregates ★
- Aggregates & subqueries ★
- Grouping ★
- NULLs with Joins, aggregates, grouping ★
- Recursion

Data Definition Language [part 3]

- views + updatable views + materialized views ★

Data Control Language

- Grant/revoke

Transaction Control Language

- begin/commit/rollback

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

Outline of this lecture

Data Query Language [part 3]

- Aggregates ★
- Aggregates & subqueries ★
- Grouping ★
- NULLs with Joins, aggregates, grouping ★
- Recursion

Data Definition Language [part 3]

- views + updatable views + materialized views ★

Data Control Language

- Grant/revoke

Transaction Control Language

- begin/commit/rollback

★ **Core Concepts: you need to understand these**

Based on chapter & online slides:

Chapters 3 and 4, Section 5.4

- from Database System Concepts 7th edition ; Silberschatz et al.

Data Control Language (DCL)

- A DBMS can have multiple users (admin, moderator, HR, ...)
- Each user or "ROLE" can access in READ/WRITE different tables or parts of it
- Grant access rights

```
GRANT select (id), update (office)
    ON professor
    TO some_user, another_user;
```

- Revoke access rights

```
REVOKE ALL PRIVILEGES
    ON professor
    FROM some_user, another_user;
```

- Privileges (rights) on tables, columns,...: select, insert, update, delete, references, **trigger**

Transaction Control Language (TCL)

- A DBMS can have multiple users operating at the same time
- An operation can take multiple QUERIES:
  1. select user salary
  2. compute MAX salary
  3. increase user salary of X% average
- What happens if someone changes the MAX salary after I run query 2?
- We start a **TRANSACTION** before query 1 and we **COMMIT** after query 3,
- nobody can change the data in the DB until I am done (or at least we wish!)
- if something goes wrong, we **ROLLBACK**

Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

Transaction Control Language (TCL)

- **Begin** a transaction

```
BEGIN;
```

- End a transaction and make all the changes made by the transaction **visible** to others and become **permanent**

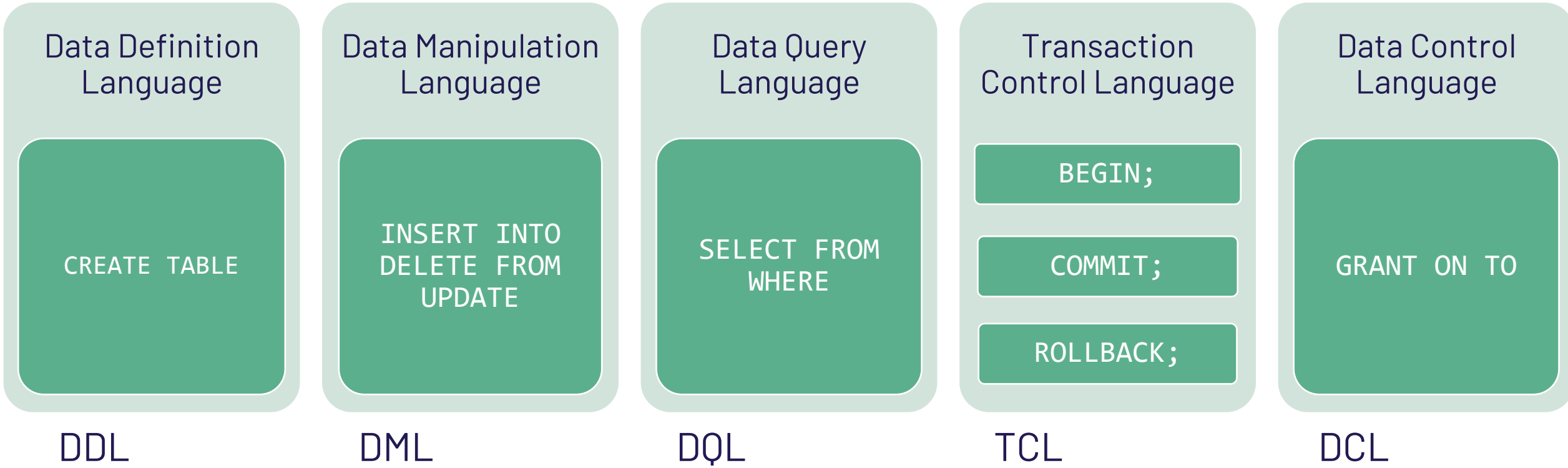
```
COMMIT;
```

- Reset a transaction and **discard** all the changes made by the transaction

```
ROLLBACK;
```


Thanks to prof. Matteo Lisandrini, Gabriela Montoya & Katja Hose

All* of SQL – Not only SELECT queries



*not really "all" , we didn't cover: WINDOW, CASE, triggers, ...
see also https://modern-sql.com/

SQL: *Declarative Query Language*

- You ask what you want not how to compute it
- Is not only about queries that retrieve data!
- Allows to handle all the aspects of “Database Management”
- Is different from the relational model and algebra because is based on “bags”